



Pedidos360

Inventario y Pedidos B2B — Informe técnico

Curso SC-601 Programación Avanzada
Universidad Fidélitas

Proyecto Final (Grupal)

Estudiantes:

UMAÑA ARAYA LUIS MARIANO
MARTINEZ BONILLA ALEJANDRO
HERNANDEZ ACEVEDO BRODIE FABRICIO
AVILA CHAVES JIMENA DE LOS ANGELES

Profesor: MONGE MORA RAUL ALEXANDER

1. Propósito del proyecto

Pedidos360 es una aplicación web desarrollada en ASP.NET Core MVC pensada para que una pyme pueda administrar su catálogo de productos, su cartera de clientes y el inventario asociado, además de armar pedidos con cálculo automático de subtotal, impuestos y descuentos. El objetivo del proyecto es demostrar el manejo de una arquitectura MVC completa: modelo de datos relacional con Entity Framework Core (Code-First), autenticación y autorización por roles con ASP.NET Identity, endpoints de apoyo consumidos vía AJAX, y manejo de errores y validaciones tanto en el cliente como en el servidor.

El sistema resuelve un problema real y acotado: hoy en día muchas pymes llevan su inventario y sus pedidos en hojas de cálculo, lo que genera errores de stock, precios desactualizados y pérdida de trazabilidad sobre quién vendió qué y cuándo. Pedidos360 centraliza esa información en una sola aplicación, con reglas de negocio (no se puede vender más stock del disponible, los totales se recalculan siempre con el precio vigente en base de datos y no con lo que envíe el navegador) y con permisos diferenciados según el rol del usuario que opera el sistema.

2. Alcance funcional

El sistema cubre el MVP solicitado en el enunciado del proyecto:

- Catálogo de productos: CRUD completo (nombre, categoría, precio, impuesto, stock, imagen, activo), con listado paginado y filtros por nombre y categoría.
- Clientes: CRUD completo con búsqueda por nombre/cédula y validaciones de correo y campos requeridos.
- Pedidos: selección de cliente, autosuggest de productos por AJAX, cálculo de subtotal/impuestos/total en tiempo real, y confirmación que descuenta stock y deja auditoría (usuario, fecha, totales).
- Seguridad: tres roles (Admin, Ventas, Operaciones) con permisos distintos sobre cada módulo, protegidos con [Authorize] a nivel de acción.
- Manejo de errores: vistas personalizadas para 403, 404 y 500, con redirección centralizada de códigos de estado.

3. Modelo de datos

El modelo quedó compuesto por seis entidades: Categoría, Producto, Cliente, Usuario (provisto por ASP.NET Identity), Pedido y PedidoDetalle. Las relaciones son todas de uno a muchos: una categoría agrupa varios productos, un cliente realiza varios pedidos, un usuario (vendedor) registra varios pedidos, y cada pedido se descompone en líneas de detalle que apuntan a un producto puntual.

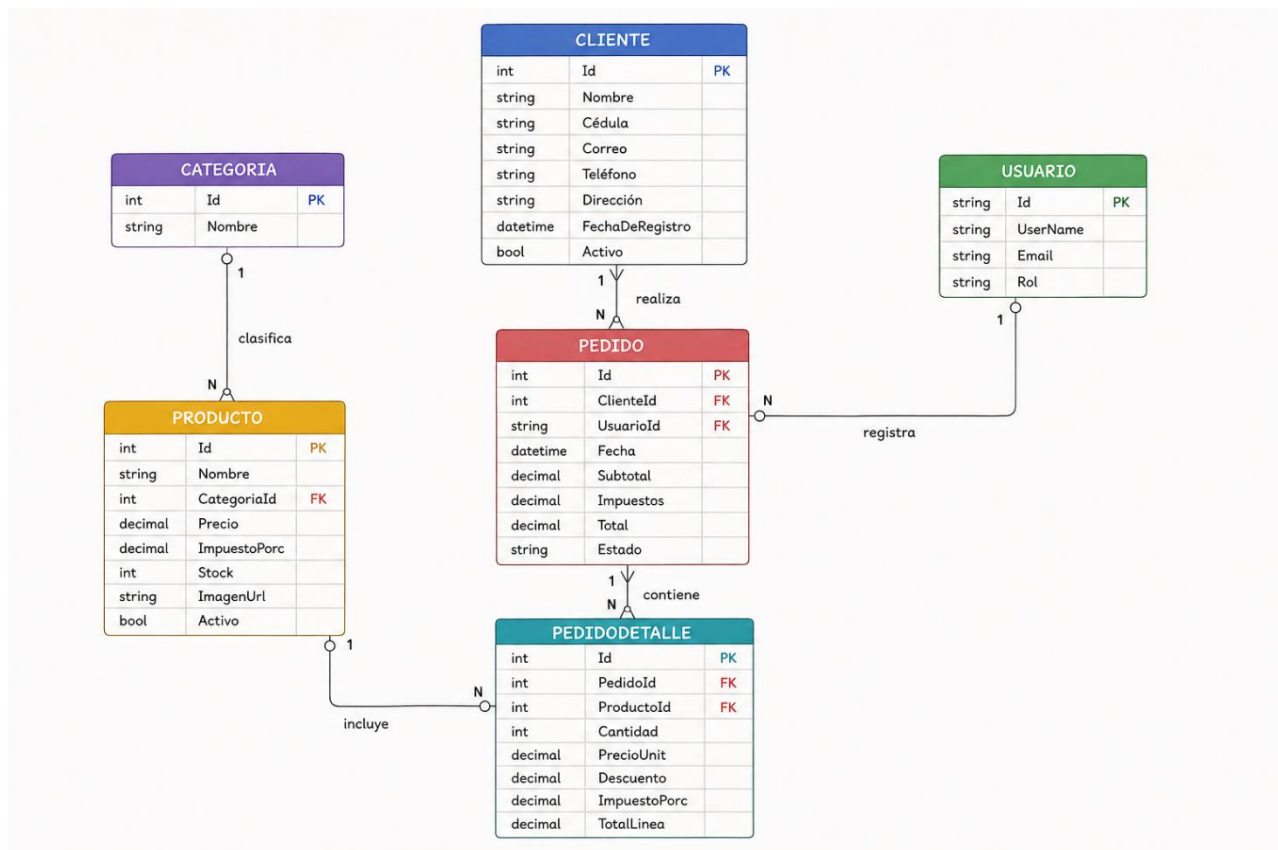


Figura 1. Diagrama entidad-relación de Pedidos360.

3.1 Justificación de normalización (3FN)

Primera forma normal: todos los atributos son atómicos. En particular, PedidoDetalle existe justamente para no repetir productos dentro de una misma fila de Pedido (evita listas dentro de una celda).

Segunda forma normal: ninguna tabla usa llave primaria compuesta, por lo que no hay dependencias parciales de las que preocuparse.

Tercera forma normal: en general ningún atributo no clave depende de otro atributo no clave. La excepción consciente son los campos de auditoría — Pedido.Subtotal/Impuestos/Total y

PedidoDetalle.PrecioUnit/ImpuestoPorc/TotalLinea — que en sentido estricto son derivables de Producto.Precio y Producto.ImpuestoPorc. Se dejaron persistidos a propósito: si el precio de un producto cambia después de confirmado un pedido, el pedido histórico debe conservar el precio y el total que efectivamente se cobraron, no el precio actual del catálogo. Es una desnormalización deliberada por trazabilidad, no un descuido del modelo.

4. Arquitectura y decisiones de diseño

La solución sigue el patrón MVC de ASP.NET Core: los Controllers consumen ApplicationDbContext (EF Core) directamente, y se apoyan en ViewModels para no exponer las entidades de dominio tal cual en las vistas. La lógica de cálculo de totales de un pedido vive en un único lugar, Services/PedidoMath.cs, compartido entre el endpoint de API (POST /api/pedidos/calcular) y el PedidosController, para garantizar que

el total que el usuario ve en pantalla mientras arma el pedido sea exactamente el mismo que se termina guardando al confirmar.

El stock se descuenta únicamente al confirmar un pedido, no al crearlo, y se repone automáticamente si un pedido confirmado se cancela después. Esto evita que pedidos abandonados a medio armar bloqueen inventario que otro cliente sí podría comprar.

5. Seguridad y roles

La autenticación se maneja con ASP.NET Core Identity y tres roles de negocio: Admin, Ventas y Operaciones. Cada acción de cada controller está anotada con `[Authorize]` o `[Authorize(Roles = "...")]` según corresponda, de forma que la restricción se aplica en el servidor y no depende de que la interfaz oculte un botón.

Acción	Admin	Ventas	Operaciones
Ver catálogo / clientes / pedidos	Sí	Sí	Sí
Crear / editar productos	Sí	No	Sí
Eliminar productos	Sí	No	No
Crear / editar clientes	Sí	Sí	No
Eliminar clientes	Sí	No	No
Crear / confirmar / cancelar pedidos	Sí	Sí	No

6. API y AJAX

El sistema expone dos endpoints protegidos con `[Authorize]`, consumidos desde el formulario de creación de pedidos:

- `GET /api/productos/buscar?q=texto` — devuelve hasta 10 productos activos con stock disponible que coinciden con el término de búsqueda, usados para el autosuggest.
- `POST /api/pedidos/calcular` — recibe las líneas que el usuario está armando (producto, cantidad, descuento) y responde con subtotal, descuentos, impuestos y total, recalculados siempre con los precios vigentes en base de datos, nunca con lo que el cliente envíe en la petición. También informa si el stock alcanza para cada línea.

Ambos endpoints requieren sesión iniciada, así que no pueden consumirse sin autenticación previa.

7. Manejo de errores

La aplicación define vistas personalizadas para los códigos de estado 403 (Forbidden), 404 (No encontrado) y 500 (Error del servidor), enrutadas mediante `UseStatusCodePagesWithReExecute`. En producción, cualquier excepción no controlada se redirige a una vista de error genérica en lugar de mostrar el stack trace, mediante `ExceptionHandler`.

8. Evidencia de cumplimiento de requisitos

Requisito del checklist	Estado
Productos: CRUD con imagen obligatoria al crear; filtros y paginación	Cumplido
Clientes: CRUD con validaciones; búsqueda por nombre/cédula	Cumplido
Pedidos: autosuggest AJAX; cálculo en vivo; persistencia de totales	Cumplido
Stock: disminuye al confirmar; no permite inventario negativo	Cumplido
Roles: Admin/Ventas/Operaciones con restricciones reales	Cumplido
Validaciones: DataAnnotations + cliente	Cumplido
API: /api/productos/buscar y /api/pedidos/calcular autenticados	Cumplido
Errores: vistas 403/404/500 personalizadas	Cumplido
DB: ER + normalización explicada (este informe)	Cumplido
README: pasos de ejecución, usuarios de prueba	Cumplido
Bono: exportar PDF/Excel, bitácora, reportes de ventas	Pendiente
Pruebas funcionales / checklist de calidad documentado	Pendiente

9. Guía de despliegue

Requisitos previos: .NET SDK 9.0 y SQL Server LocalDB.

- `git clone https://github.com/eelizondogene-rgb/Pedido360.git`
- `cd Pedido360/Pedido360`
- `dotnet restore`
- `dotnet ef database update`
- `dotnet run`

Al iniciar por primera vez, DbInitializer siembra automáticamente los tres roles del sistema con un usuario de prueba por rol (admin@pedido360.com / Admin123!, ventas@pedido360.com / Ventas123!, operaciones@pedido360.com / Operaciones123!), 8 clientes de ejemplo y 12 productos distribuidos en 4 categorías.

10. Conclusiones y mejoras futuras

El proyecto cumple el MVP obligatorio completo: catálogo, clientes, pedidos con cálculo en vivo, seguridad por roles y manejo de errores. Quedan pendientes, como trabajo futuro y para optar al bono de la rúbrica, la exportación de pedidos a PDF/Excel, una bitácora de auditoría de cambios, reportes básicos de ventas por fecha o cliente, y la posibilidad de subir la imagen del producto como archivo en lugar de una URL externa.